



Customer 360 Data Integration

Bootcamp Project - 3



APRIL 30, 2025
BY HARJINDER SINGH

Table of Contents

Introduction	2
Raw Data Ingestion (Bronze Layer).....	3
Raw Data Transformation to Staging (Silver Layer)	4
Verify the Output (Staging Layer)	16
Create Views in Azure SQL DB (Gold Layer)	19
Power BI Report	28

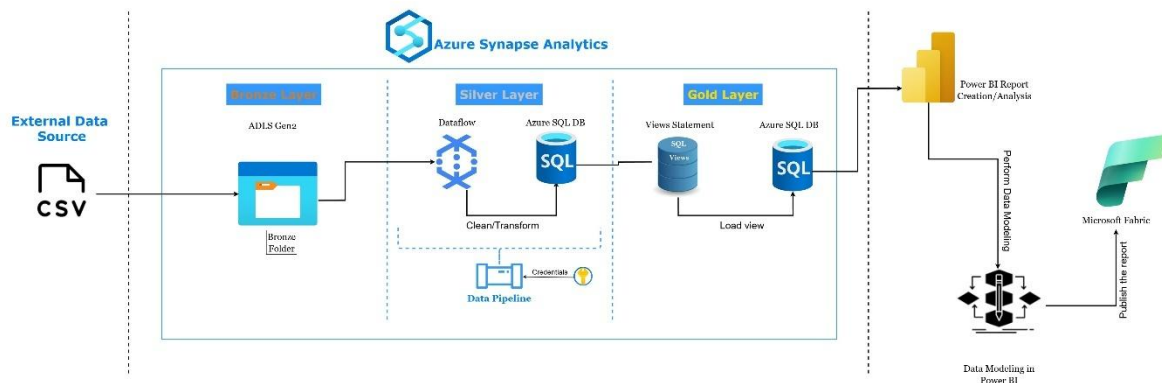
Introduction

In this project, I integrated data from multiple retail sources—online transactions, in-store purchases, customer service interactions, and loyalty programs—to build a unified Customer 360 view that supports better business decision-making.

Using **Azure Synapse**, **Azure Data Lake Storage (ADLS)**, **Azure SQL Database**, and **Power BI**, I designed and implemented a scalable, end-to-end data pipeline. The data flows through three layers: **Raw**, **Curated (Silver)**, and **Analytics (Gold)**, enabling structured and meaningful insights.

I created views for KPIs (Key Performance Indicators) such as Average Order Value, Customer Segmentation, Peak Purchase Times, and Agent Performance. The final output is a dynamic **Power BI dashboard** that delivers actionable insights for sales, marketing, and customer support teams.

The following **architecture** will be used for this Project.



Raw Data Ingestion (Bronze Layer)

Using Azure Synapse Analytics Data pipelines to ingest data from multiple sources (online, in-store, customer service, loyalty programs) into the raw container in ADLS.

We will download dataset from below link on local Storage.

Dataset source: <https://www.kaggle.com/datasets/varunkumari/customer-360-data>

Create a folder called Bronze_Layer in ADLS Gen 2 storage account and upload all the csv files from the Local storage.

Home > hsinghadls | Containers >

project3-container

Container

Search

Upload Add Directory Refresh Rename Delete Change tier Acquire lease Break lease Give feedback

Location: project3-container / Bronze_Layer

Search blobs by prefix (case-sensitive)

Show deleted objects

Overview

Diagnose and solve problems

Access Control (IAM)

Settings

	Name	Modified	Access tier	Archive status	Blob type	Size	Lease state	
☐	[.]							...
☐	Agents.csv	4/29/2025, 2:01:06 PM	Hot (Inferred)		Block blob	3.82 KiB	Available	...
☐	Customers.csv	4/29/2025, 2:01:06 PM	Hot (Inferred)		Block blob	8.72 KiB	Available	...
☐	CustomerService...	4/29/2025, 2:01:06 PM	Hot (Inferred)		Block blob	5 KiB	Available	...
☐	InStoreTransactio...	4/29/2025, 2:01:06 PM	Hot (Inferred)		Block blob	4.62 KiB	Available	...
☐	LoyaltyAccounts...	4/29/2025, 2:01:06 PM	Hot (Inferred)		Block blob	2.93 KiB	Available	...
☐	LoyaltyTransactio...	4/29/2025, 2:01:07 PM	Hot (Inferred)		Block blob	3.67 KiB	Available	...
☐	OnlineTransactio...	4/29/2025, 2:01:07 PM	Hot (Inferred)		Block blob	5.43 KiB	Available	...
☐	Products.csv	4/29/2025, 2:01:07 PM	Hot (Inferred)		Block blob	2.57 KiB	Available	...
☐	Stores.csv	4/29/2025, 2:01:07 PM	Hot (Inferred)		Block blob	4.84 KiB	Available	...

Raw Data Transformation to Staging (Silver Layer)

Step 1: Open SSMS and Connect the Azure SQL Database. Create the below tables in this sequence (as they have constraints).

```
CREATE TABLE Customers (  
    CustomerID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Email VARCHAR(100),  
    Address VARCHAR(255)  
);
```

```
CREATE TABLE Products (  
    ProductID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Category VARCHAR(50),  
    Price DECIMAL(10, 2)  
);
```

```
CREATE TABLE OnlineTransactions (  
    OrderID INT PRIMARY KEY,  
    CustomerID INT,  
    ProductID INT,  
    DateTime DATETIME,  
    PaymentMethod VARCHAR(50),  
    Amount DECIMAL(10, 2),  
    Status VARCHAR(20),  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),  
    FOREIGN KEY (ProductID) REFERENCES Products(ProductID)  
);
```

```
CREATE TABLE Stores (  
    StoreID INT PRIMARY KEY,  
    Location VARCHAR(100),  
    Manager VARCHAR(100),  
    OpenHours VARCHAR(50)  
);
```

```
CREATE TABLE InStoreTransactions (  
    TransactionID INT PRIMARY KEY,  
    CustomerID INT,  
    StoreID INT,  
    DateTime DATETIME,  
    Amount DECIMAL(10, 2),
```

```
PaymentMethod VARCHAR(50),  
FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),  
FOREIGN KEY (StoreID) REFERENCES Stores(StoreID)  
);
```

```
CREATE TABLE Agents (  
    AgentID INT PRIMARY KEY,  
    Name VARCHAR(100),  
    Department VARCHAR(50),  
    Shift VARCHAR(50)  
);
```

```
CREATE TABLE CustomerServiceInteractions (  
    InteractionID INT PRIMARY KEY,  
    CustomerID INT,  
    DateTime DATETIME,  
    AgentID INT,  
    IssueType VARCHAR(50),  
    ResolutionStatus VARCHAR(50),  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),  
    FOREIGN KEY (AgentID) REFERENCES Agents(AgentID)  
);
```

```
CREATE TABLE LoyaltyAccounts (  
    LoyaltyID INT PRIMARY KEY,  
    CustomerID INT,  
    PointsEarned INT,  
    TierLevel VARCHAR(20),  
    JoinDate DATE,  
    FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)  
);
```

```
CREATE TABLE LoyaltyTransactions (  
    LoyaltyID INT,  
    DateTime DATETIME,  
    PointsChange INT,  
    Reason VARCHAR(100),  
    PRIMARY KEY (LoyaltyID, DateTime),  
    FOREIGN KEY (LoyaltyID) REFERENCES LoyaltyAccounts(LoyaltyID)  
);
```

```

CREATE TABLE Customers (
  CustomerID INT PRIMARY KEY,
  Name VARCHAR(100),
  Email VARCHAR(100),
  Address VARCHAR(255)
);

CREATE TABLE Products (
  ProductID INT PRIMARY KEY,
  Name VARCHAR(100),
  Category VARCHAR(50),
  Price DECIMAL(10, 2)
);

CREATE TABLE OnlineTransactions (
  OrderID INT PRIMARY KEY,
  CustomerID INT,
  ProductID INT,
  DateTime DATETIME,
  PaymentMethod VARCHAR(50),
  Amount DECIMAL(10, 2),
  Status VARCHAR(20),
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
  FOREIGN KEY (ProductID) REFERENCES Products(ProductID)
);

CREATE TABLE Stores (
  StoreID INT PRIMARY KEY,
  Location VARCHAR(100),
  Manager VARCHAR(100),
  OpenHours VARCHAR(50)
);

CREATE TABLE InStoreTransactions (
  TransactionID INT PRIMARY KEY,
  CustomerID INT,
  StoreID INT,
  DateTime DATETIME,
  Amount DECIMAL(10, 2),
  PaymentMethod VARCHAR(50),
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
  FOREIGN KEY (StoreID) REFERENCES Stores(StoreID)
);

CREATE TABLE Agents (
  AgentID INT PRIMARY KEY,
  Name VARCHAR(100),
  Department VARCHAR(50),
  Shift VARCHAR(50)
);

CREATE TABLE CustomerServiceInteractions (
  InteractionID INT PRIMARY KEY,
  CustomerID INT,
  DateTime DATETIME,
  AgentID INT,
  IssueType VARCHAR(50),
  ResolutionStatus VARCHAR(50),
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID),
  FOREIGN KEY (AgentID) REFERENCES Agents(AgentID)
);

CREATE TABLE LoyaltyAccounts (
  LoyaltyID INT PRIMARY KEY,
  CustomerID INT,
  PointsEarned INT,
  TierLevel VARCHAR(20),
  JoinDate DATE,
  FOREIGN KEY (CustomerID) REFERENCES Customers(CustomerID)
);

CREATE TABLE LoyaltyTransactions (
  LoyaltyID INT,
  DateTime DATETIME,
  PointsChange INT,
  Reason VARCHAR(100),
  PRIMARY KEY (LoyaltyID, DateTime),
  FOREIGN KEY (LoyaltyID) REFERENCES LoyaltyAccounts(LoyaltyID)
);

```

Step 2: Open Synapse Workspace -> Create a new pipeline -> Drag and drop three dataflows' activities in canvas area.

- Click on the first one, and Settings -> Dataflow -> New
- Blank dataflow design canvas will be opened
- Here, we will design transformation step from bronze to silver (staging) layer

pl_data_transform... x df_parent_tables_tra... df_child_tables_trans... df_child_tables_trans...

Other users in your workspace may have access trust all users who may have access to the works

Activities < Search activities

- > Synapse
- > Move and transform
- > Azure Data Explorer
- > Azure Function
- > Batch Service
- > Databricks

Validate Debug Add trigger Data flow debug

Data flow df_parent_tables_tr ansformation

Data flow df_child_tables_tran sformation

Data flow df_child_tables_tran sformation2

Step 3: Click on Add source -> Click on Source Type -> Inline and select the source.

Validate Data flow debug

Agents Columns: 4 total

Source settings Source options Projection Optimize Inspect Data preview

Output stream name * Agents [Learn more](#)

Description Import data from wsp-hsingh-synapse-WorkspaceDefaultStorage [Reset](#)

Source type * Integration dataset **Inline** Workspace DB

Inline dataset type * DelimitedText

Linked service * wsp-hsingh-synapse-WorkspaceDefa... [Test connection](#) [Edit](#) [New](#)

Skip line count

Sampling * ☐ Enable ☒ Disable

- Next, in the Source option, select the file path.
- And checked First row as header option.

Source settings **Source options** Projection Optimize Inspect Data preview

File mode ☒ File ☐ Wildcard

File path * / / [Browse](#)

Allow no files found ☐

Change data capture ☐

Compression type

Encoding

Column delimiter

Row delimiter

Quote character

Escape character

First row as header ☒

Step 4: Click on Projection tab and import schema.

Source settings Source options **Projection** Optimize Inspect Data preview

← Import schema ✕ Clear schema 📄 Schema options

Column name	Type	Format
AgentID	12s short	Specify format
Name	abc string	Specify format
Department	abc string	Specify format
Shift	abc string	Specify format

Step 5: Add the filter transformation to remove all null records.

– Expression Used:

!isNull(AgentID)

✓ Validate ☐ Data flow debug

Filter settings Optimize Inspect Data preview

Output stream name * [Learn more](#)

Description [Reset](#)

Incoming stream *

Filter on *

Step 6: Add the Aggregate transformation to remove all duplicates records.

- Here, In Group by Option section, I have selected all column to remove duplicates based on all column match value.

✓ Validate ☐ Data flow debug

Agents
Import data from
wsp-hsingh-synapse-Works
paceDefaultStorage

Reference:
1
Columns:
4 total

RemoveDuplicateRows
Columns:
5 total

Aggregate settings Optimize Inspect Data preview

Output stream name * RemoveDuplicateRows [Learn more](#)

Description
Aggregating data by 'AgentID, Name, Department, Shift' producing columns 'Count' [Reset](#)

Incoming stream * DropNullRows

Group by Aggregates

Columns	Name as		
12s AgentID	AgentID	+	🗑️
abc Name	Name	+	🗑️
abc Department	Department	+	🗑️
abc Shift	Shift	+	🗑️

- In Aggregate Option section, I have added a dummy column name count to perform the count aggregate function and using that function aggregating the unique records.

Group by Aggregates

Grouped by: AgentID, Name, Department, Shift

+ Add [Clone](#) [Delete](#) [Open expression builder](#)

Column	Expression
☐ Count	count(1) 121 + 🗑️

Step 7: Add the Select transformation to remove dummy column which is created in last transformation.

✓ Validate ☒ Data flow debug

Agents Import data from DropNullRows Filtering rows using Reference: 1 DropDummyCol

Select settings Optimize Inspect Data preview

Description Renaming RemoveDuplicateRows to DropDummyCol with columns 'AgentID, Name, Department, Shift' Reset

Incoming stream * RemoveDuplicateRows

Options ☒ Skip duplicate input columns ☒ Skip duplicate output columns

Input columns * ☐ Auto mapping ☒ Reset + Add mapping Delete 4 mappings: 1 column(s) from the inputs left unmapped

RemoveDuplicateRows's column	Name as
12s AgentID	AgentID
abc Name	Name
abc Department	Department
abc Shift	Shift

Step 8: Add the derived column transformation to replace the null value in the column with a meaningful value.

Expressions used:

- `iif(isNull(trim(Name)), 'Unknown', Name)`
- `iif(isNull(trim(Department)), 'N/A', Department)`
- `iif(isNull(trim(Shift)), 'Unknown', Shift)`

✓ Validate ☒ Data flow debug

Agents Import data from wsp-hsingh-synapse-WorkspaceDefaultStorage DropNullRows Filtering rows using expressions on columns 'AgentID' Reference: 1 Columns: 5 total DropDummyCol Renaming RemoveDuplicateRows to DropDummyCol with columns 'AgentID, Name, Department, Shift' FillNullValues Columns: 4 total

Derived column's settings Optimize Inspect Data preview

Description Creating/updating the columns 'AgentID, Name, Department, Shift' Reset

Incoming stream * DropDummyCol

+ Add Clone Delete Open expression builder

Columns * ☐ Column Expression

☐ Name	<code>iif(isNull(trim(Name)), 'Unknown', Name)</code>	abc	+ ☐
☐ Department	<code>iif(isNull(trim(Department)), 'N/A', Department)</code>	abc	+ ☐
☐ Shift	<code>iif(isNull(trim(Shift)), 'Unknown', Shift)</code>	abc	+ ☐

Step 9: Add alter row sink transformation to give Upsert Permission on SQL DB table

- Alter row conditions: **Upsert if**

- Expression: $1==1$

The screenshot shows the configuration for the 'GiveUpsertPermissions' node. The top bar displays the data flow pipeline: RemoveDuplicateRows → DropDummyCol → FillNullValues → GiveUpsertPermissions → AzureSQLDBAgents. The 'GiveUpsertPermissions' node is selected, showing 4 total columns. Below the pipeline, the configuration options are:

- Output stream name ***: GiveUpsertPermissions
- Description**: Add expressions to alter rows
- Incoming stream ***: FillNullValues
- Alter row conditions ***: Upsert if $1==1$

Step 10: Create sink, select Azure SQL DB dataset type.

- Click on New to create new Linked services.

The screenshot shows the configuration for the 'AzureSQLDBAgents' node. The top bar displays the data flow pipeline: DropDummyCol → FillNullValues → GiveUpsertPermissions → AzureSQLDBAgents. The 'AzureSQLDBAgents' node is selected, showing 4 total columns. Below the pipeline, the configuration options are:

- Output stream name ***: AzureSQLDBAgents
- Description**: Add sink dataset
- Incoming stream ***: GiveUpsertPermissions
- Sink type ***: Integration dataset, **Inline**, Workspace DB, Cache
- Inline dataset type ***: Azure SQL Database
- Linked service ***: Select... **+ New**
- Options**: ☒ Allow schema drift, ☐ Validate schema

- While Creating Linked Service, use Azure Key vault option to store Azure SQL server password in to key vault.

Edit linked service

Azure SQL Database [Learn more](#)

Account selection method ^①

☐ From Azure subscription ☒ Enter manually

Fully qualified domain name *

hsinghsqlserver.database.windows.net

Database name *

hsinghsqldb

Authentication type *

SQL authentication

User name *

hsingh2025

AKV linked service * ^①

Is_AzureKeyVault

Secret name *

azure-sqldb-password

☒ Edit

Secret version ^①

Latest version

☐ Edit

Always encrypted ^① ☐

Encrypt ^①

Mandatory

- For this we need to create key vault linked service as well, Go to Key Vault and click on Generate/Import. Write Secret name and Secret value (password of Azure SQL Server) and click on Create.

Home > Key vaults > hsinghkeyvault

hsinghkeyvault | Secrets

Key vault

Search

Access control (IAM)

Tags

Diagnose and solve problems

Access policies

Resource visualizer

Events

Objects

Keys

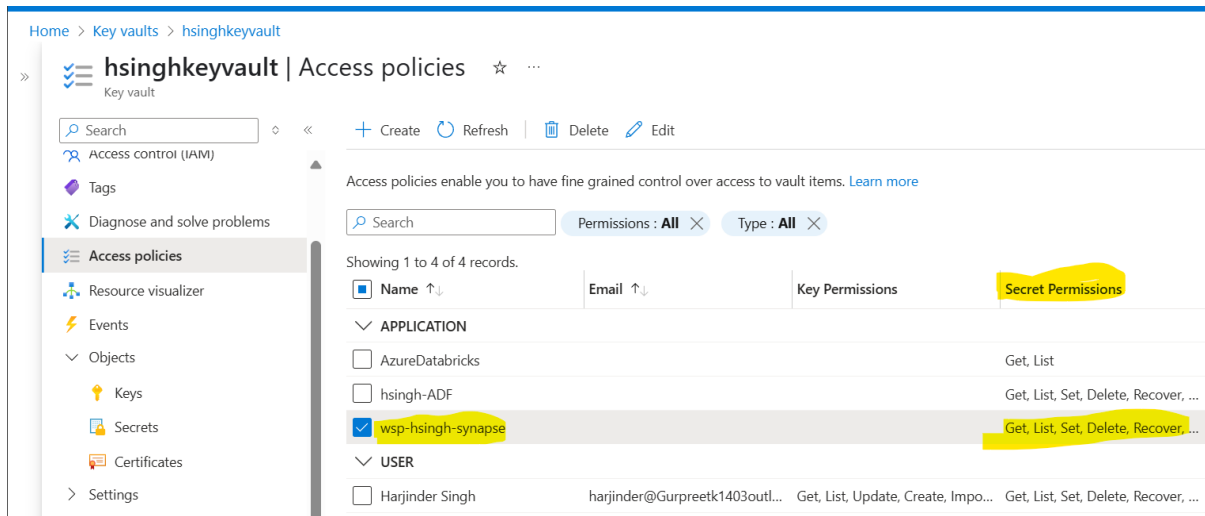
Secrets

Certificates

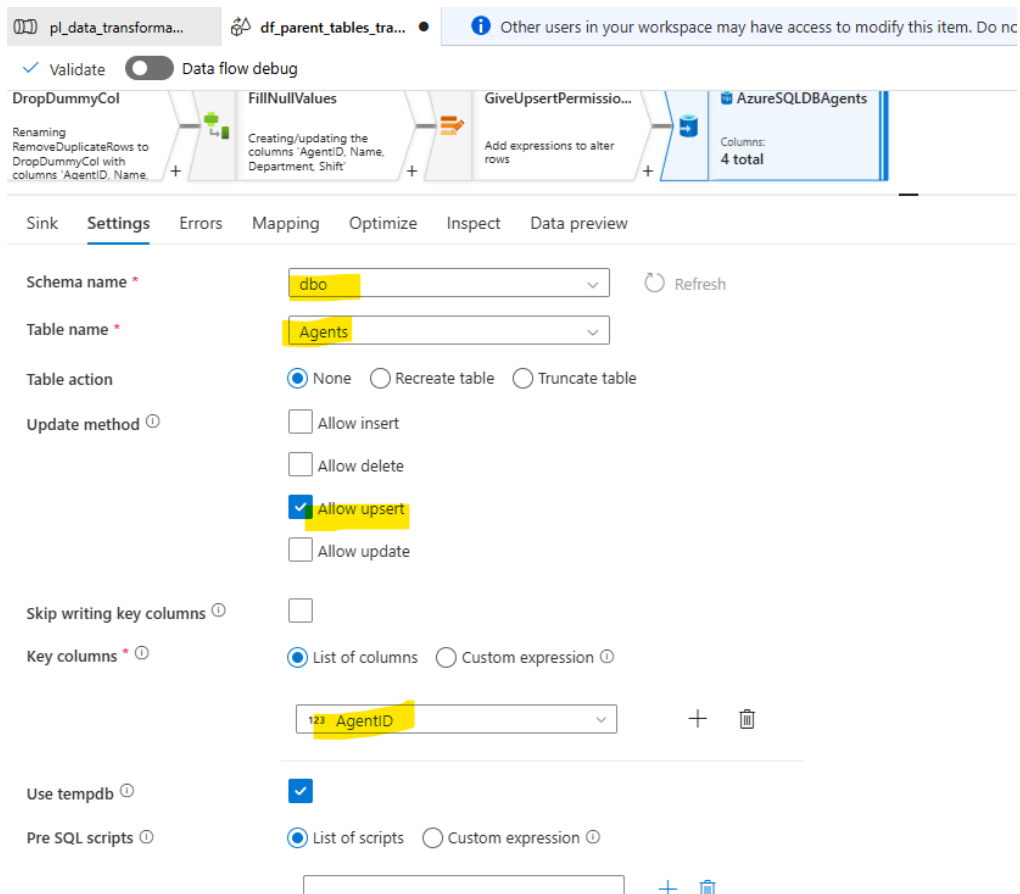
Generate/Import Refresh Restore Backup Manage deleted secrets View sample code

Name	Type	Status
appid		✓ Enabled
appsecret		✓ Enabled
azure-sqldb-password		✓ Enabled
azure-sqldb-username		✓ Enabled
onprem-system-password		✓ Enabled

- Also, in Key Vault Access Policies give permission (Get, List for Synapse Workspace to access key vault secrets)



- Next Go to Settings tab of sink, Schema Name: dbo (select from a dropdown list)
 - Table Name: Agents (select from a dropdown list)
 - Update Method: **Allow Upsert**
 - Key Columns: AgentID

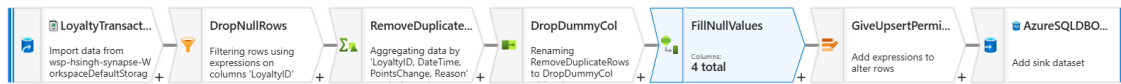


- Go to Mapping tab -> Click on Import Schema and map the columns as shown below:

Now, the data transformation is finished for **Agents** Data file.

****And similarly, we need to implement the similar transformation for rest of the parent tables (Customers, Products, and Stores) to clean the data and store it in Silver layer Azure SQL DB tables.**

For the child table also, we need to implement the similar transformation for rest of the child tables (CustomerServiceInteractions, InStoreTransactions, LoyaltyAccounts, LoyaltyTransactions, and OnlineTransactions) to clean the data and store it in Silver layer Azure SQL DB tables.



Add Source | ▾

Verify the Output (Staging Layer)

Verify the output after running the dataflows pipeline using the below queries:

```
SELECT * FROM Customers;
SELECT * FROM Products;
SELECT * FROM OnlineTransactions;
SELECT * FROM Stores;
SELECT * FROM InStoreTransactions;
SELECT * FROM Agents;
SELECT * FROM CustomerServiceInteractions;
SELECT * FROM LoyaltyAccounts;
SELECT * FROM LoyaltyTransactions;
```

Results		Messages		
	CustomerID	Name	Email	Address
1	1	Mrs. Crystal Carroll	kaiserjacob@example.org	7566 Kelly Shoals Apt. 207, Port Joanne, FM 25401
2	2	Debra Newman ...	brownashley@example....	72713 Nelson Lodge Suite 286, Leonfort, NJ 49406
3	3	Thomas Mason	frankmark@example.com	3024 Riley Ferry Suite 573, Chadberg, MT 40822
4	4	Karla Hill	jnelson@example.org	9899 Hubbard Station, Lake Vanessa, CT 97851
5	5	Jeffrey Underwo...	daniel48@example.org	11764 Hannah Plaza, Lake Vickichester, AR 85240
6	6	Justin Lowe	qwilson@example.org	494 Danielle Causeway, Lake Benjamin, NV 21217
7	7	Jason Daniels	elliottsonia@example.net	37283 Ramsey Light Suite 640, New Howard, IA ...
8	8	Jennifer Hill	petersenrebecca@exa...	02411 Ruiz Corners Apt. 346, Oconnorberg, SD 9...
9	9	Stephanie Richa...	obates@example.net	974 Alyssa Heights, Farrellfurt, OH 22597
10	10	Jo Zimmerman	samuel23@example.net	25747 Clark Harbor, New Emilyport, RI 59995
11	11	Andrew Williams	robinsonjulian@examl...	431 Thomas Orchard, South Aliciaside, ME 26651

	ProductID	Name	Category	Price
1	1	Consider	Electronics	183.51
2	2	At	Toys	403.28
3	3	Risk	Clothing	186.53
4	4	Serious	Clothing	275.39
5	5	Discover	Clothing	23.39
6	6	Area	Books	316.84
7	7	Happy	Toys	137.58
8	8	Effect	Electronics	202.63
9	9	Star	Books	119.08
10	10	Structu...	Books	273.71
11	11	Join	Books	253.70
12	12	Course	Books	298.30
13	13	Play	Home G...	478.47

Results Messages							
	OrderID	CustomerID	ProductID	DateTime	PaymentMethod	Amount	Status
1	1	34	91	2025-01-20 22:34:31.000	Gift Card	189.90	Completed
2	2	17	59	2025-02-10 20:42:13.000	Gift Card	193.25	Pending
3	3	13	57	2025-02-16 18:26:04.000	Debit Card	183.82	Pending
4	4	90	19	2025-02-21 05:04:59.000	Debit Card	175.37	Pending
5	5	61	99	2025-01-19 05:42:37.000	Credit Card	174.62	Completed
6	6	75	67	2025-02-26 09:47:16.000	Debit Card	144.48	Pending
7	7	63	44	2025-02-07 09:48:20.000	PayPal	91.74	Failed
8	8	55	6	2025-01-19 20:32:41.000	Credit Card	16.58	Failed

	StoreID	Location	Manager	OpenHours
1	1	Angelamouth	Michelle Robinson	8:00 AM - 8:00 PM
2	2	Tammychester	Jonathan Frey	8:00 AM - 8:00 PM
3	3	Vanessache...	Barbara Lynch	8:00 AM - 7:00 PM
4	4	Maytown	Frederick Dean	9:00 AM - 8:00 PM
5	5	Kathleenfort	Donald Ellis	10:00 AM - 7:00 ...
6	6	South Martin	Tamara Hernan...	10:00 AM - 6:00 ...
7	7	Rebeccastad	Amanda Diaz	9:00 AM - 8:00 PM
8	8	North Mariab...	Lisa Jackson	9:00 AM - 7:00 PM
9	9	Stephenfurt	Michael Hays	10:00 AM - 9:00 ...
10	10	Thompsonsh...	Mary Barker	9:00 AM - 9:00 PM
11	11	East Justin	Jose Brown	9:00 AM - 8:00 PM
12	12	Jeffreyport	Roger Walsh	8:00 AM - 9:00 PM
13	13	New James	Crystal Choi	10:00 AM - 9:00 ...
14	14	Josephfort	Ryan King	8:00 AM - 6:00 PM
15	15	New Kimberly	Caleb Powers	8:00 AM - 6:00 PM
16	16	Lake Phillip	Nathan Turner	8:00 AM - 6:00 PM

Results Messages						
	TransactionID	CustomerID	StoreID	DateTime	Amount	PaymentMethod
1	1	88	100	2025-03-12 06:49:37.000	135.74	Credit Card
2	2	60	12	2025-01-21 23:09:14.000	34.14	Credit Card
3	3	54	51	2025-01-29 22:12:51.000	111.42	Debit Card
4	4	32	94	2025-02-08 23:51:36.000	179.45	Debit Card
5	5	33	45	2025-01-18 04:05:21.000	177.96	Gift Card
6	6	92	91	2025-03-09 03:29:17.000	196.51	Debit Card
7	7	44	10	2025-02-19 19:28:55.000	198.12	Debit Card
8	8	91	51	2025-02-15 14:38:42.000	19.48	Gift Card
9	9	76	77	2025-01-19 09:16:43.000	196.53	Debit Card
10	10	65	12	2025-01-30 06:43:44.000	83.99	Gift Card
11	11	13	77	2025-01-29 21:46:00.000	180.81	Credit Card
12	12	89	89	2025-01-15 18:29:30.000	88.37	Credit Card

	AgentID	Name	Department	Shift
1	1	Jonathan Williams	General Inquiry	Morning
2	2	Terry Edwards	Billing	Evening
3	3	Garrett Knapp	Sales	Morning
4	4	Daryl Benjamin	Sales	Evening
5	5	Matthew Long	Sales	Morning
6	6	Patricia Rhodes	Sales	Morning
7	7	Elizabeth James	Technical Su...	Morning
8	8	Teresa Bennett	Technical Su...	Morning
9	9	Amber Ross	Billing	Aftern...
10	10	Tonya Jones	Technical Su...	Evening
11	11	Curtis McBride	Sales	Aftern...
12	12	Justin Michael	Billing	Morning

Results Messages						
	InteractionID	CustomerID	DateTime	AgentID	IssueType	ResolutionStatus
1	1	55	2025-01-20 08:17:26.000	33	Technical Issue	Resolved
2	2	93	2025-01-16 18:52:51.000	89	Technical Issue	Escalated
3	3	53	2025-03-16 13:10:05.000	96	Complaint	Resolved
4	4	30	2025-02-27 13:45:03.000	20	Other	Pending
5	5	49	2025-02-16 07:56:45.000	48	Technical Issue	Pending
6	6	42	2025-02-13 21:05:41.000	56	Product Inquiry	Resolved
7	7	73	2025-03-01 13:19:50.000	38	Other	Escalated
8	8	50	2025-01-15 16:44:36.000	66	Other	Escalated
9	9	63	2025-03-11 10:58:12.000	32	Complaint	Pending
10	10	21	2025-03-15 05:00:10.000	78	Product Inquiry	Escalated
11	11	69	2025-01-03 21:23:28.000	4	Complaint	Escalated
12	12	85	2025-02-17 10:30:24.000	34	Billing	Escalated

	LoyaltyID	CustomerID	PointsEarned	TierLevel	JoinDate
1	1	5	399	Gold	2023-11-27
2	2	17	606	Gold	2021-06-26
3	3	13	4782	Gold	2023-04-11
4	4	62	1824	Platinum	2020-12-17
5	5	84	894	Gold	2020-03-18
6	6	68	4291	Platinum	2020-09-25
7	7	90	2845	Bronze	2022-09-26
8	8	51	1697	Silver	2022-04-14
9	9	5	1912	Silver	2023-07-03
10	10	69	3846	Bronze	2024-09-28
11	11	53	2155	Silver	2022-04-24
12	12	86	4396	Bronze	2020-11-01

Results Messages				
	LoyaltyID	DateTime	PointsChange	Reason
1	1	2025-01-13 20:04:21.000	166	Purchase
2	2	2025-02-28 18:14:23.000	190	Promotion
3	2	2025-03-04 21:11:47.000	-30	Promotion
4	6	2025-01-12 22:56:22.000	29	Referral
5	8	2025-02-21 03:46:32.000	122	Referral
6	9	2025-02-24 08:12:28.000	186	Correction
7	10	2025-01-04 02:14:30.000	60	Correction
8	10	2025-02-25 21:36:08.000	179	Referral
9	12	2025-01-03 14:46:28.000	182	Correction
10	12	2025-01-06 18:33:47.000	58	Referral
11	13	2025-01-09 07:30:47.000	172	Promotion
12	14	2025-01-17 01:13:54.000	108	Referral
13	15	2025-03-15 21:15:07.000	-34	Referral
14	17	2025-01-28 03:04:00.000	183	Referral
15	17	2025-02-18 09:27:33.000	-6	Purchase
16	18	2025-01-27 02:16:47.000	139	Promotion
17	18	2025-02-27 03:25:47.000	159	Purchase
18	20	2025-03-15 14:39:12.000	-33	Promotion
19	21	2025-02-03 01:13:19.000	39	Purchase
20	21	2025-02-07 17:27:02.000	82	Promotion
21	21	2025-02-07 23:37:42.000	-10	Promotion
22	21	2025-02-17 14:58:30.000	171	Purchase
23	22	2025-01-31 01:31:22.000	178	Referral
24	22	2025-02-23 12:50:05.000	6	Promotion
25	24	2025-01-06 14:57:11.000	25	Correction

Create Views in Azure SQL DB (Gold Layer)

1. View 1 - for Average Order Value (AOV)

SUM(Amount) / COUNT(OrderID) per product, category, and location.

```
CREATE VIEW View1_AverageOrderValue AS
SELECT
    p.Category AS ProductCategory,
    c.Address AS Location,
    COUNT(o.OrderID) AS TotalOnlineOrders,
    SUM(o.Amount) AS Amount,
    CAST(ROUND(SUM(o.Amount) / COUNT(o.OrderID), 2) AS DECIMAL(10, 2)) AS
AverageOrderValue
FROM
    dbo.OnlineTransactions AS o
JOIN
    dbo.Customers AS c ON o.CustomerID = c.CustomerID
JOIN
    dbo.Products AS p ON o.ProductID = p.ProductID
GROUP BY
    p.Category, c.Address

SELECT * FROM View1_AverageOrderValue ORDER BY Amount DESC
```

<pre> CREATE VIEW View1_AverageOrderValue AS SELECT p.Category AS ProductCategory, c.Address AS Location, COUNT(o.OrderID) AS TotalOnlineOrders, SUM(o.Amount) AS Amount, CAST(ROUND(SUM(o.Amount) / COUNT(o.OrderID), 2) AS DECIMAL(10, 2)) AS AverageOrderValue FROM dbo.OnlineTransactions AS o JOIN dbo.Customers AS c ON o.CustomerID = c.CustomerID JOIN dbo.Products AS p ON o.ProductID = p.ProductID GROUP BY p.Category, c.Address SELECT * FROM View1_AverageOrderValue ORDER BY Amount DESC </pre>					
74 %					
Results Messages					
	ProductCategory	Location	TotalOnlineOrders	Amount	AverageOrderValue
1	Books	711 Smith Drives, Port David, PW 40381	2	335.87	167.94
2	Clothing	634 Thomas Lights, Gonzalezland, IL 49739	2	295.12	147.56
3	Clothing	482 Harris Mission, Adamsland, UT 89317	2	211.99	106.00
4	Clothing	75190 John Ferry Apt. 305, North Renee, FL 93070	1	199.99	199.99
5	Books	0574 Johnson Ford Apt. 404, Hayesside, NC 59305	1	195.76	195.76
6	Home Goods	754 Hunt Trafficway, Port Laura, NH 04226	1	193.25	193.25
7	Home Goods	265 Yates Pike, Tammyton, WA 87644	1	192.49	192.49
8	Home Goods	84152 Smith Track, Chelseastad, HI 83035	1	189.50	189.50
9	Books	447 Daniel Glens, Kentmouth, PA 58193	1	187.76	187.76
10	Books	431 Thomas Orchard, South Aliciaside, ME 26651	1	187.64	187.64
11	Electronics	839 Gates Forges East Catherineville MS 93946	1	186.36	186.36

2. View 2 - for Segment customers based on total spend, purchase frequency, and loyalty tier (LoyaltyAccounts.TierLevel).

Example: "High-Value Customers" (Top 10% spenders), "One-Time Buyers," "Loyalty Champions."

```
CREATE View View2_CutsomerSegments AS
```

```
WITH LoyaltyRanked AS (
```

```
    SELECT
```

```
        la.CustomerID,
```

```
        la.TierLevel,
```

```
        CASE la.TierLevel
```

```
            WHEN 'Platinum' THEN 4
```

```
            WHEN 'Gold' THEN 3
```

```
            WHEN 'Silver' THEN 2
```

```
            WHEN 'Bronze' THEN 1
```

```
            ELSE 0
```

```
        END AS TierRank
```

```
    FROM LoyaltyAccounts la
```

```
),
```

```

HighestTier AS (
    SELECT CustomerID, TierLevel
    FROM (
        SELECT *,
            ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY TierRank
DESC) AS rnk
        FROM LoyaltyRanked
    ) ranked
    WHERE rnk = 1
),
OnlineSpend AS (
    SELECT
        CustomerID,
        SUM(Amount) AS OnlineAmount,
        COUNT(DISTINCT OrderID) AS PurchaseCount
    FROM OnlineTransactions
    GROUP BY CustomerID
),
InStoreSpend AS (
    SELECT
        CustomerID,
        SUM(Amount) AS InStoreAmount,
        COUNT(DISTINCT TransactionID) AS PurchaseCount
    FROM InStoreTransactions
    GROUP BY CustomerID
),
CustomerSpend AS (
    SELECT
        c.CustomerID,
        c.Name,
        c.Email,
        ISNULL(ht.TierLevel, 'None') AS TierLevel,
        ISNULL(os.OnlineAmount, 0) + ISNULL(iss.InStoreAmount, 0) AS TotalSpend,
        ISNULL(os.PurchaseCount, 0) + ISNULL(iss.PurchaseCount, 0) AS
PurchaseFrequency
    FROM Customers c
    LEFT JOIN HighestTier ht ON c.CustomerID = ht.CustomerID
    LEFT JOIN OnlineSpend os ON c.CustomerID = os.CustomerID
    LEFT JOIN InStoreSpend iss ON c.CustomerID = iss.CustomerID
),
SpendWithPercentile AS (

```

```

SELECT *,
    PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY TotalSpend) OVER () AS
P90Spend
FROM CustomerSpend
)
SELECT
    CustomerID,
    Name,
    Email,
    TierLevel,
    TotalSpend,
    PurchaseFrequency,
    CASE
        WHEN TotalSpend >= P90Spend THEN 'High-Value Customer'
        WHEN PurchaseFrequency = 1 THEN 'One-Time Buyer'
        WHEN TierLevel IN ('Gold', 'Platinum') THEN 'Loyalty Champion'
        ELSE 'Regular Customer'
    END AS Segment
FROM SpendWithPercentile;

SELECT * FROM View2_CutsomerSegments ORDER BY TotalSpend DESC;

```

```

CREATE View View2_CutsomerSegments AS
WITH LoyaltyRanked AS (
    SELECT
        la.CustomerID,
        la.TierLevel,
        CASE la.TierLevel
            WHEN 'Platinum' THEN 4
            WHEN 'Gold' THEN 3
            WHEN 'Silver' THEN 2
            WHEN 'Bronze' THEN 1
            ELSE 0
        END AS TierRank
    FROM LoyaltyAccounts la
),
HighestTier AS (
    SELECT CustomerID, TierLevel
    FROM (
        SELECT *,
            ROW_NUMBER() OVER (PARTITION BY CustomerID ORDER BY TierRank DESC) AS rnk
        FROM LoyaltyRanked
    ) ranked
    WHERE rnk = 1
),
OnlineSpend AS (
    SELECT
        CustomerID,
        SUM(Amount) AS OnlineAmount,
        COUNT(DISTINCT OrderID) AS PurchaseCount
    FROM OnlineTransactions
    GROUP BY CustomerID
),
InStoreSpend AS (
    SELECT
        CustomerID,
        SUM(Amount) AS InStoreAmount,
        COUNT(DISTINCT TransactionID) AS PurchaseCount
    FROM InStoreTransactions
    GROUP BY CustomerID
),
CustomerSpend AS (
    SELECT
        c.CustomerID,
        c.Name,
        c.Email,
        ISNULL(ht.TierLevel, 'None') AS TierLevel,
        ISNULL(os.OnlineAmount, 0) + ISNULL(iss.InStoreAmount, 0) AS TotalSpend,
        ISNULL(os.PurchaseCount, 0) + ISNULL(iss.PurchaseCount, 0) AS PurchaseFrequency
    FROM Customers c
    LEFT JOIN HighestTier ht ON c.CustomerID = ht.CustomerID
    LEFT JOIN OnlineSpend os ON c.CustomerID = os.CustomerID
    LEFT JOIN InStoreSpend iss ON c.CustomerID = iss.CustomerID
),
SpendWithPercentile AS (
    SELECT *,
        PERCENTILE_CONT(0.9) WITHIN GROUP (ORDER BY TotalSpend) OVER () AS P90Spend
    FROM CustomerSpend
)
SELECT
    CustomerID,
    Name,
    Email,
    TierLevel,
    TotalSpend,
    PurchaseFrequency,
    CASE
        WHEN TotalSpend >= P90Spend THEN 'High-Value Customer'
        WHEN PurchaseFrequency = 1 THEN 'One-Time Buyer'
        WHEN TierLevel IN ('Gold', 'Platinum') THEN 'Loyalty Champion'
        ELSE 'Regular Customer'
    END AS Segment
FROM SpendWithPercentile;

SELECT * FROM View2_CutsomerSegments ORDER BY TotalSpend DESC;

```


	Results	Messages					
	CustomerID	Name	Email	TierLevel	TotalSpend	PurchaseFrequency	Segment
1	26	Michelle Wood	michaelgraham@example.com	Gold	770.81	5	High-Value Customer
2	73	Linda Boyd	brendafisher@example.net	Bronze	716.37	5	High-Value Customer
3	74	Edward Lee	mossbeverly@example.org	None	621.74	4	High-Value Customer
4	18	Alisha Phillips	lrobinson@example.org	Silver	595.73	4	High-Value Customer
5	44	Elizabeth Taylor	ijones@example.org	Platinum	557.23	4	High-Value Customer
6	33	Robert Lopez	xgriffin@example.org	None	540.84	4	High-Value Customer
7	13	Gina Anderson	michelle40@example.com	Gold	512.43	3	High-Value Customer
8	68	Amy Horton	kim95@example.com	Platinum	462.00	3	High-Value Customer
9	47	Stephanie Carrillo	sarah68@example.org	Gold	460.85	4	High-Value Customer
10	39	John Cook	mdavis@example.org	Silver	459.22	6	High-Value Customer
11	20	Luis Jones	gabrielatkins@example.net	Gold	448.54	4	Loyalty Champion
12	71	Nathan Long	nathanwest@example.net	None	427.08	3	Regular Customer
13	91	Rachel Lewis	james42@example.org	Silver	411.15	4	Regular Customer
14	90	Katherine Mcca...	tanya69@example.net	Bronze	395.93	3	Regular Customer
15	82	Amanda Torres	gonzalesjennifer@example.org	None	393.54	3	Regular Customer
16	63	Tony Jenkins D...	robert49@example.com	Gold	366.97	3	Loyalty Champion
17	97	Joel Jensen	cconley@example.com	Silver	359.80	3	Regular Customer
18	12	Robin Odom	deborah35@example.com	Bronze	359.28	3	Regular Customer
19	49	Austin Marshall	jasminegeorge@example.com	Bronze	354.69	4	Regular Customer
20	75	Nicholas Morales	igreene@example.net	Platinum	354.66	3	Loyalty Champion
21	66	Sara Ford	michaeltgarcia@example.net	None	353.91	3	Regular Customer

3. View 3 - for Analyze DateTime to find peak days and times in-store vs. online. CREATE VIEW View3_PeakTimeTransactions AS

```

SELECT
    'Online' AS Channel,
    DATENAME(WEEKDAY, DateTime) AS DayOfWeek,
    DATEPART(HOUR, DateTime) AS HourOfDay,
    COUNT(OrderID) AS TotalTransactions
FROM
    OnlineTransactions
GROUP BY
    DATENAME(WEEKDAY, DateTime),
    DATEPART(HOUR, DateTime)

```

UNION ALL

```

SELECT
    'In-Store' AS Channel,
    DATENAME(WEEKDAY, DateTime) AS DayOfWeek,
    DATEPART(HOUR, DateTime) AS HourOfDay,
    COUNT(TransactionID) AS TotalTransactions
FROM
    InStoreTransactions
GROUP BY
    DATENAME(WEEKDAY, DateTime),
    DATEPART(HOUR, DateTime);

```

SELECT * FROM View3_PeakTimeTransactions

```
CREATE VIEW View3_PeakTimeTransactions AS
SELECT
    'Online' AS Channel,
    DATENAME(WEEKDAY, DateTime) AS DayOfWeek,
    DATEPART(HOUR, DateTime) AS HourOfDay,
    COUNT(OrderID) AS TotalTransactions
FROM
    OnlineTransactions
GROUP BY
    DATENAME(WEEKDAY, DateTime),
    DATEPART(HOUR, DateTime)

UNION ALL

SELECT
    'In-Store' AS Channel,
    DATENAME(WEEKDAY, DateTime) AS DayOfWeek,
    DATEPART(HOUR, DateTime) AS HourOfDay,
    COUNT(TransactionID) AS TotalTransactions
FROM
    InStoreTransactions
GROUP BY
    DATENAME(WEEKDAY, DateTime),
    DATEPART(HOUR, DateTime);

SELECT * FROM View3_PeakTimeTransactions
```

SELECT * FROM View3_PeakTimeTransactions				
89 %				
Results Messages				
	Channel	DayOfWeek	HourOfDay	TotalTransactions
9	Online	Wednesday	2	1
10	Online	Saturday	3	2
11	Online	Sunday	3	1
12	Online	Tuesday	3	1
13	Online	Friday	4	1
14	Online	Thursday	4	2
15	Online	Tuesday	4	1
16	Online	Wednesday	4	2
17	Online	Friday	5	3
18	Online	Sunday	5	1
19	Online	Tuesday	5	1
20	Online	Wednesday	5	2
21	Online	Monday	6	1
22	Online	Sunday	6	1
23	Online	Thursday	6	1
24	Online	Wednesday	6	1
25	Online	Friday	7	1
26	Online	Tuesday	7	2
27	Online	Monday	8	1
28	Online	Saturday	8	2
29	Online	Wednesday	8	1

4. View 4 - for Number of interactions and resolution success rates per agent (**ResolutionStatus**).

```

CREATE VIEW View4_ResolutionSuccessRate AS
SELECT
    a.AgentID,
    a.Name AS AgentName,
    a.Department,
    COUNT(csi.InteractionID) AS TotalInteractions,
    SUM(CASE WHEN csi.ResolutionStatus = 'Resolved' THEN 1 ELSE 0 END) AS
ResolvedInteractions,
    CAST(ROUND(
        100.0 * SUM(CASE WHEN csi.ResolutionStatus = 'Resolved' THEN 1 ELSE 0 END)
        / NULLIF(COUNT(csi.InteractionID), 0), 2
    ) AS DECIMAL(5,2)) AS ResolutionSuccessRate
FROM
    Agents a
LEFT JOIN
    CustomerServiceInteractions csi ON a.AgentID = csi.AgentID
GROUP BY

```

a.AgentID, a.Name, a.Department;

SELECT * FROM View4_ResolutionSuccessRate

```
CREATE VIEW View4_ResolutionSuccessRate AS
SELECT
    a.AgentID,
    a.Name AS AgentName,
    a.Department,
    COUNT(csi.InteractionID) AS TotalInteractions,
    SUM(CASE WHEN csi.ResolutionStatus = 'Resolved' THEN 1 ELSE 0 END) AS ResolvedInteractions,
    CAST(ROUND(
        100.0 * SUM(CASE WHEN csi.ResolutionStatus = 'Resolved' THEN 1 ELSE 0 END)
        / NULLIF(COUNT(csi.InteractionID), 0), 2
    ) AS DECIMAL(5,2)) AS ResolutionSuccessRate
FROM
    Agents a
LEFT JOIN
    CustomerServiceInteractions csi ON a.AgentID = csi.AgentID
GROUP BY
    a.AgentID, a.Name, a.Department;

SELECT * FROM View4_ResolutionSuccessRate
```

SELECT * FROM View4_ResolutionSuccessRate

89 %

Results

Messages

	AgentID	AgentName	Department	TotalInteractions	ResolvedInteractions	ResolutionSuccessRate
1	1	Jonathan Williams	General Inquiry	1	0	0.00
2	2	Terry Edwards	Billing	1	0	0.00
3	3	Garrett Knapp	Sales	0	0	NULL
4	4	Daryl Benjamin	Sales	2	0	0.00
5	5	Matthew Long	Sales	1	0	0.00
6	6	Patricia Rhodes	Sales	0	0	NULL
7	7	Elizabeth James	Technical Support	2	2	100.00
8	8	Teresa Bennett	Technical Support	1	0	0.00
9	9	Amber Ross	Billing	2	1	50.00
10	10	Tonya Jones	Technical Support	1	0	0.00
11	11	Curtis McBride	Sales	1	1	100.00
12	12	Justin Michael	Billing	0	0	NULL
13	13	Scott Flowers	Billing	1	0	0.00
14	14	Michael Jackson	Technical Support	0	0	NULL
15	15	Kristen Crawford	Sales	1	1	100.00
16	16	Jo Meyers	General Inquiry	0	0	NULL
17	17	Brandon Jimenez	Technical Support	1	0	0.00
18	18	Melissa White	General Inquiry	2	0	0.00
19	19	Eddie Pierce	Sales	1	0	0.00
20	20	Julia Owens	Sales	1	0	0.00
21	21	Cindy Gomez	Technical Support	3	2	66.67

Power BI Report

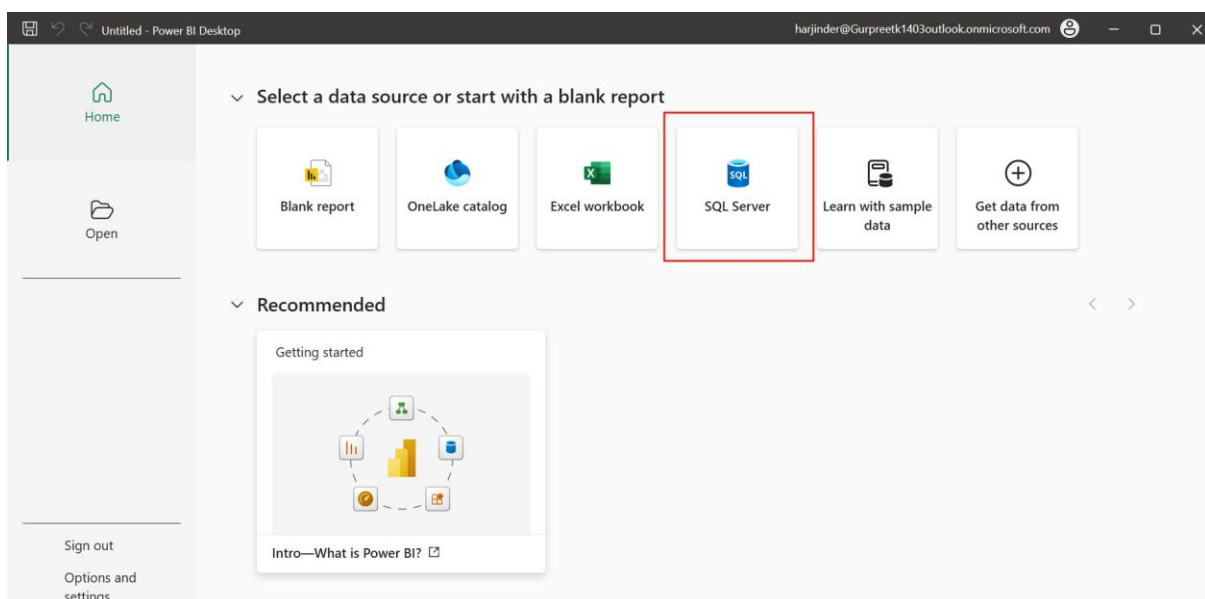
Here, I have used Power BI desktop Application to create and publish the report using gold Layer data i.e., 4 views from Azure SQL DB.

- View1_AverageOrderValue
- View2_CutsomerSegments
- View3_PeakTimeTransactions
- View4_ResolutionSuccessRate

➤ Steps To Create and Publish the Power BI report in Microsoft Fabric workspace.

Step 1: Install and open the Power BI desktop applications -> login with your current working azure portal credentials. (make sure the same credentials are used to login in Fabric account in order to access workspace).

Next, select SQL server as data Source.



Step 2: Next, enter the SQL server name (you can find it in Azure SQL DB overview page in Azure Portal) -> select Import as Data Connectivity mode. -> click on OK.

SQL Server database

Server ⓘ

hsinghsqserver.database.windows.net

Database (optional)

Data Connectivity mode ⓘ

☒ Import
 ☐ DirectQuery

▶ Advanced options

OK

Cancel

Step 3: Next, select the data Views from the db -> click on Load.

Navigator

Display Options ▼

hsinghsqserver.database.windows.net [1]

hsinghsqldb [38]

☐ CustomerSegmentation
 ☐ SalesLT.vGetAllCategories
 ☐ SalesLT.vProductAndDescription
 ☐ SalesLT.vProductModelCatalogDescri...
 ☐ sys.database_firewall_rules
 ☒ View1_AverageOrderValue
 ☒ View2_CutsomerSegments
 ☒ View3_PeakTimeTransactions
 ☒ View4_ResolutionSuccessRate
 ☐ accounts
 ☐ Agents
 ☐ BuildVersion
 ☐ Customers
 ☐ CustomerServiceInteractions
 ☐ emp
 ☐ ErrorLog
 ☐ InStoreTransactions

View3_PeakTimeTransactions

Channel	DayOfWeek	HourOfDay	TotalTransactions
Online	Monday	0	1
Online	Sunday	0	1
Online	Tuesday	0	1
Online	Friday	1	1
Online	Saturday	1	1
Online	Tuesday	1	1
Online	Monday	2	2
Online	Saturday	2	1
Online	Wednesday	2	1
Online	Saturday	3	2
Online	Sunday	3	1
Online	Tuesday	3	1
Online	Friday	4	1
Online	Thursday	4	2
Online	Tuesday	4	1
Online	Wednesday	4	2
Online	Friday	5	3
Online	Sunday	5	1
Online	Tuesday	5	1
Online	Wednesday	5	2
Online	Monday	6	1
Online	Sunday	6	1
Online	Thursday	6	1

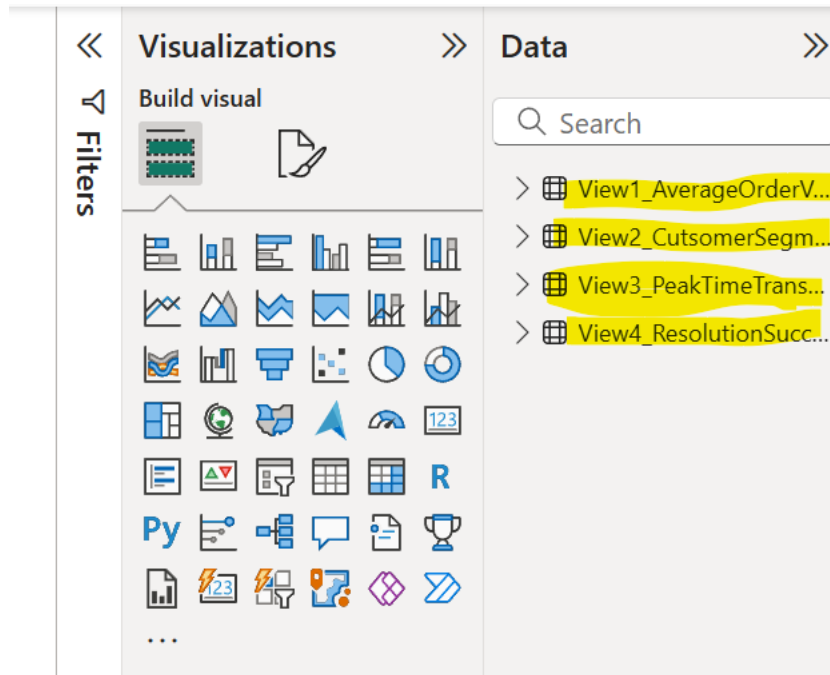
Select Related Tables

Load

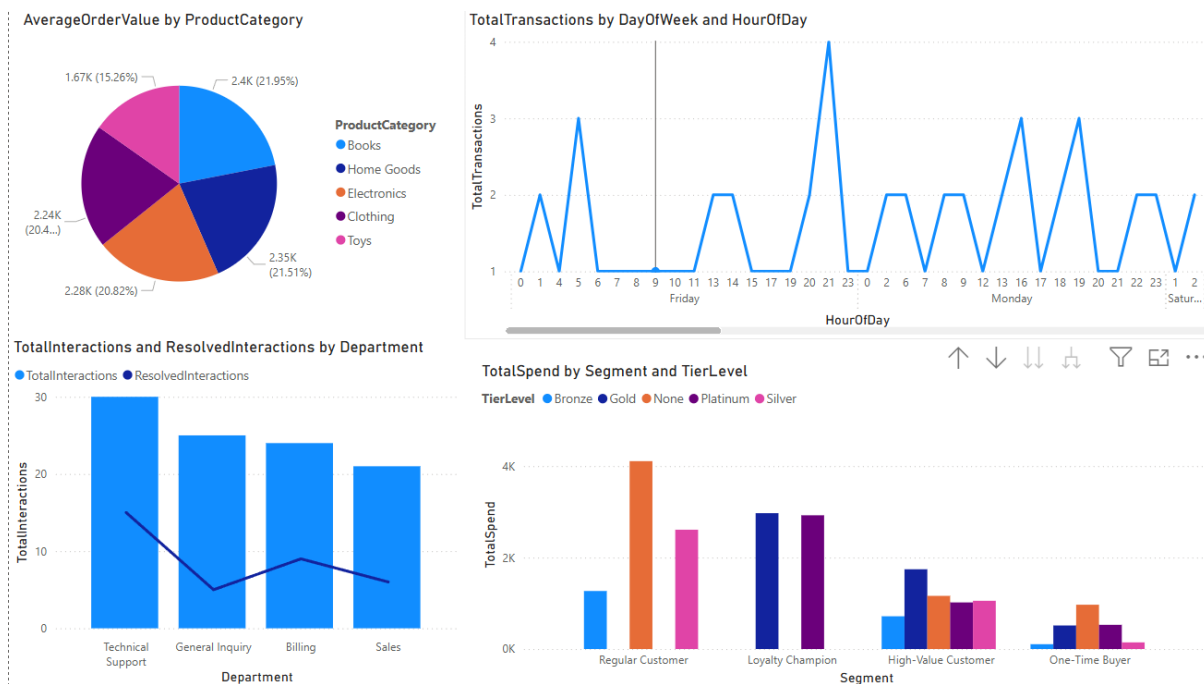
Transform Data

Cancel

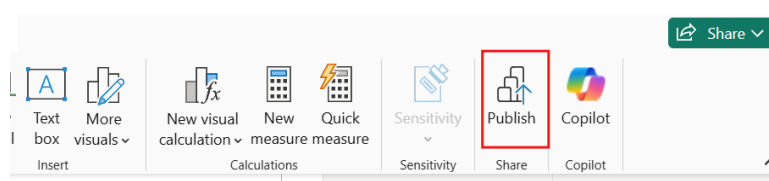
Step 4: Next, select the data columns to visualize the data from data section.



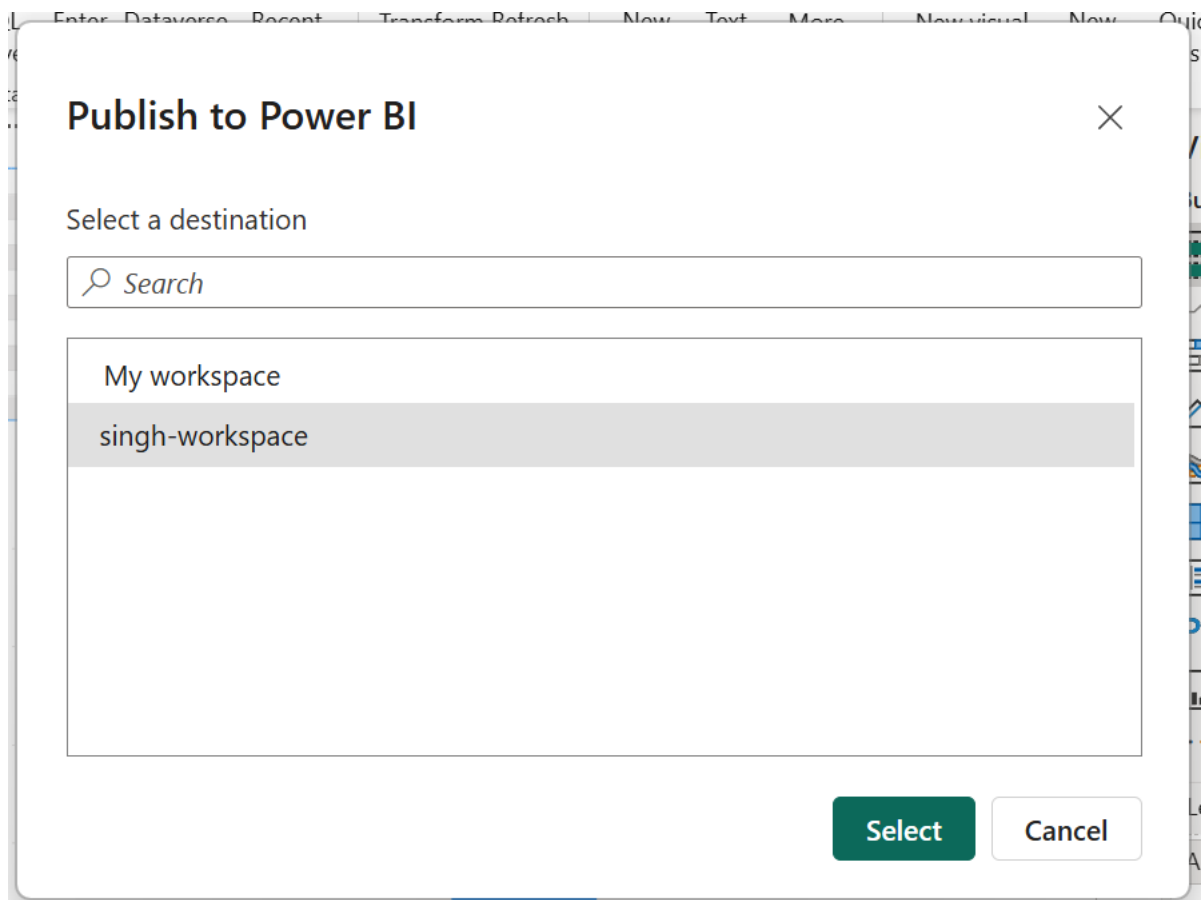
Step 5: Next, create some visuals and report by using data columns.



Step 6: Next, review the report and publish using the publish option above.



Step 7: Next, save the report and select the fabric workspace to publish the power BI report -> click on select.



Step 8: Finally, review the published report in the fabric workspace.

Name	Type	Task	Owner	Refreshed	Next refresh	Endorsement	Sensitivity	Included in app
Bootcamp Project-3 Report	Report	—	singh-work...	5/1/2025, 6:25...	—	—	—	No
Bootcamp Project-3 Report	Semantic ...	—	singh-work...	5/1/2025, 6...	N/A	—	—	
myDatabricksProjectReport	Report	—	singh-work...	4/24/2025, 5:4...	—	—	—	No
myDatabricksProjectReport	Semantic ...	—	singh-work...	4/24/2025, 5...	N/A	—	—	
MyProjectReport	Report	—	singh-work...	4/22/2025, 4:4...	—	—	—	No
MyProjectReport	Semantic ...	—	singh-work...	4/22/2025, ...	N/A	—	—	
singhLakehouse	Lakehouse	—	Harjinder S...	—	—	—	—	
singhLakehouse	Semantic ...	—	singh-work...	4/19/2025, 3...	N/A	—	—	
singhLakehouse	SQL analyti...	—	Harjinder S...	—	—	—	—	